

Problem 1: Unaprijediti JSON parser sa vježbi

Na vježbama je rađen parser za json format koji ima određene propuste koji su namjerno izostavljeni. Vaš zadatak je da kroz objašnjenja ispod, kompletirate parser. Za rješenje zadatka potrebno je modifikovati polaznu gramatiku, koja je data u fileu rules.txt. Programski kod je implementiran RDP pristupom i on mora odgovarati gramatici u rules.txt fileu.

Kao polazni kalup za ovaj projekat možete uzeti json primjer sa prethodnih vježbi.

Prvi dio Kod rješenja sa vježbi, nije moguće postaviti prazan json objekat. Sljedeći ulaz mora biti tačan.

```
{ }
```

Dodatni zarez nije dozvoljen po specifikaciji JSON-a. Testirati na Vašem rješenju sljedeća dva nedozvoljena ulaza:

- Zarez poslije "foo":

```
{ a : 1, b : "foo", }
```

- Sam zarez:

```
{,}
```

Drugi dio Jsn gramatika poznaje pojam liste vrijednosti. Primjeri validnog ulaza:

```
{ a : [] }
```

```
{ a : [1, 2, 3, "foo", true, false, null, { "id": 1} ] }
```

Treći dio Po json specifikaciji, top-level pravilo može biti objekat ili lista. Tako da su validni i json ulazi koji odmah počinju sa listom.

```
[ { "item": "mlijeko" }, { "item" : "kafa" }, { "item" : "secer" } ]  
[ true , { "id" : 320 }, 15 ]
```

Napomena: obično web servisi zabranjuju top level json formatu da sadrži listu radi određenih sigurnosnih problema JavaScripta.

Problem 2: Gramatika za podskup Lox jezika

Vaš zadatak je da osmislite gramatiku u BNF notaciji za pojednostavljen podskup Lox jezika. Jedan Lox program je sačinjen od naredbi (**statement**) koje se ponavljaju, dok jedna naredba može biti:

1. **Deklaracija varijable bez inicijalizacije:** `var x;`
2. **Deklaracija varijable sa inicijalizacijom:** `var y = 42 + x;`
3. **Izraz (expression):** `x = 5 + 4 * 2 == 15 - 4 / 2;`

4. **Definicija funkcije:** `fun ime(parametri) { tijelo }`
5. **If-izraz (bez else grane):** `if (uslov) { naredbe }`

Zahtjevi

Gramatika treba da poštuje sljedeća pravila:

- Funkcija može imati nula ili više parametara (razdvojenih zarezom)
- Tijelo funkcije i if-bloka sastoji se od niza naredbi (**statement**) unutar vitičastih zagrada
- Uslov u if-izrazu je proizvoljan izraz (**expression**)
- Imena varijabli i funkcija su identifikatori po pravilu kao u C jeziku
- Jedini tip podatka je cijeli broj (pozitivan ili negativan),

Operatori i prioriteti Izrazi (**expression**) treba da sadrže sljedeće operatore organizovane po prioritetu (od najnižeg ka najvišem):

1. **Operator dodjeljivanja** (=) - najniži prioritet
2. **Relacioni operatori** (==, !=, <, >) - svi istog prioriteta
3. **Aditivni operatori** (+, -)
4. **Multiplikativni operatori** (*, /) - najviši prioritet

Napomene: - nije potrebno obezbijediti ponavljanje operatora=, (npr. nije potrebno omogućiti `a = b = 4 * 2;`), - sa lijeve strane operatora= se mora naći identifikator, - operator= se može javiti u if-izrazu.

Primjer programa

```
var counter;
var limit = 10;

fun increment(x) {
    var result = x + 1;
    if (result < limit) {
        counter = result;
    }
}

fun calculate(a, b) {
    var sum = a + b;
    var product = a * b;
    var comparison = sum == product;
    if (sum > 100) {
        result = sum / 2;
    }
}

var fahr;
```

```
var celz = 100;
fahr = celz * 9 / 5 + 32;
```

Napomene za izraze (**expression**)

- Svaki nivo prioriteta treba predstaviti zasebnim neterminalom, kako bi bilo moguće napisati RDP parser,
- Operatori istog prioriteta mogu se grupisati u isti neterminal
- Gramatika podržava male zagrade

Smjernice za rješenje

1. Počnite od **program** koji je niz **statement**-a
2. Definirajte različite tipove **statement**-a (deklaracija, funkcija, if, itd.)
3. Za izraze, kreirajte hijerarhiju neterminala:
 - **assignment** (najniži prioritet)
 - **equality** (relacioni operatori)
 - **term** (sabiranje i oduzimanje)
 - **factor** (množenje i dijeljenje)
 - **primary** (literali i identifikatori - najviši prioritet)

Problem 3: RDP parser za Lox jezik

Koristeći gramatiku iz prošlog zadatka, potrebno je implementirati parser za Lox jezik. Kao lekser možete iskoristiti lekser iz zadatke 2.

Predaja zadatke

Putem classrooma predajete arhivu sa nazivom **ime_prezime_zadaca4.tar.gz**. Otpakovana arhiva treba sadržavati jedan folder imena **ime_prezime_zadaca4** u kojem se nalazi Vaše rješenje. Prvi problem u jednom folderu, a drugi i treću u jednom. Gramatiku iz drugog zadatka dati u fileu sa imenom **rules.txt**.

Komandom **make** potrebno je kompajlirati projekat u prvom i trećem zadatku. Izvršnji fileovi u prvom i trećem zadatku trebaju imati imena **jpase** i **loxpase**.

Ne predajete binarne fileove, kao npr. **a.out**.